Routing System Design and Algorithm Analysis

1. Introduction

The purpose of this project is to implement a routing engine capable of finding efficient paths inside a road network. The road network is modeled as a weighted graph where intersections are represented as vertices and roads are represented as edges.

Unlike a traditional shortest path problem that only considers distance, this project also considers traffic conditions, weather conditions, and the delay at each destination node. Therefore, the routing algorithm attempts to find the path with the minimum overall travel cost rather than the minimum physical distance.

The total traversal cost of an edge is defined as

$$Cost(u,v)=Distance(u,v)\times Traffic(u,v)\times Weather(u,v)+Delay(v)$$

where:

- Distance represents the road length.
- Traffic represents the traffic congestion coefficient.
- Weather represents the road/weather condition coefficient.
- Delay(v) represents the waiting time at the destination intersection.

The implementation automatically selects the appropriate shortest path algorithm depending on whether negative edge weights exist.

2. Graph Representation

The road network is stored as an adjacency list.

Each edge stores:

- Source node
- Destination node
- Distance
- Traffic coefficient
- Weather coefficient

Each node stores:

- Node ID
- Delay value

The graph is implemented as

map<int, vector>

while node delays are stored in

unordered_map<int, double>

Why an adjacency list?

An adjacency list is more suitable than an adjacency matrix because road networks are generally sparse graphs. Most intersections are connected to only a small number of neighboring intersections.

Advantages

- Requires less memory
- Efficient traversal of neighboring nodes
- Faster for sparse graphs

Disadvantages

- Slower when checking whether a specific edge exists
- Slightly more complex implementation than an adjacency matrix

Complexity

Operation	Time	Space
Build graph	O(V + E)	O(V + E)

3. File Loading

The project reads two CSV files.

Nodes file

Contains:

- Node ID
- Delay

Edges file

Contains:

- Source
- Destination
- Distance
- Traffic
- Weather

During loading, every edge is converted into its total routing cost and the program checks whether any edge becomes negative.

If a negative edge exists, Bellman-Ford will later be used instead of Dijkstra.

4. Dijkstra Algorithm

Purpose

Dijkstra's algorithm is used to compute the shortest path between two nodes when all edge weights are non-negative.

The implementation uses a priority queue (min-heap) to always expand the node with the currently smallest distance.

For every neighboring edge,

newCost = currentDistance + edgeCost + destinationDelay

If the new cost is smaller than the previously recorded distance, the distance and parent are updated.

Why Dijkstra was chosen

Most real road networks contain only positive travel costs.

When all edge weights are positive, Dijkstra is one of the fastest shortest-path algorithms available.

It is significantly faster than Bellman-Ford for large graphs.

Time Complexity

Using a priority queue,

Time

O((V+E)log V)

Space

O(V+E)

Advantages

- Very fast
- Efficient on large sparse graphs
- Finds the optimal shortest path
- Well suited for road networks

Disadvantages

- Cannot handle negative edge weights
- Produces incorrect results if negative edges exist

5. Bellman-Ford Algorithm

Purpose

Bellman-Ford is used whenever at least one edge has a negative weight.

Unlike Dijkstra, Bellman-Ford repeatedly relaxes every edge until no shorter path can be found.

After all relaxations, one additional iteration is performed.

If a shorter path is still found, the graph contains a negative cycle.

In this situation, no valid shortest path exists because the total cost can decrease indefinitely by repeatedly traversing the negative cycle.

The implementation returns a special value to indicate this condition.

Why Bellman-Ford was chosen

Bellman-Ford guarantees correct shortest paths even when negative edge weights are present.

It also provides negative cycle detection, which is required by the project specification.

Time Complexity

Time

O(VE)

Space

O(V)

Advantages

- Handles negative edge weights correctly
- Detects negative cycles
- Always produces correct results when no negative cycle exists

Disadvantages

- Much slower than Dijkstra
- Not suitable for very large graphs unless necessary

6. Automatic Algorithm Selection

Instead of forcing the user to choose an algorithm manually, the routing system automatically selects the appropriate algorithm.

The selection process is simple:

- If all edge weights are non-negative → Dijkstra
- If at least one negative edge exists → Bellman-Ford

This approach combines the speed of Dijkstra with the correctness of Bellman-Ford.

7. Parent Array

Both shortest-path algorithms maintain a parent array.

Whenever a better path is found,

parent[destination] = currentNode

This allows the shortest route to be reconstructed after the algorithm finishes by tracing the parent pointers backward from the destination to the source.

8. Nearest Neighbor Routing

Problem

The project also requires visiting several destinations before returning to the starting point.

This is a simplified version of the Travelling Salesman Problem (TSP).

Finding the exact optimal solution for TSP is computationally expensive because the problem is NP-hard.

Therefore, this project uses the Nearest Neighbor heuristic.

Main Idea

The algorithm begins at the starting node.

At each step,

1. Compute the shortest path cost to every unvisited destination.
2. Select the closest destination.
3. Move to that destination.
4. Remove it from the unvisited set.
5. Repeat until every destination has been visited.
6. Finally return to the starting node.

The shortest path between two destinations is computed using the routing algorithm already selected (Dijkstra or Bellman-Ford).

Why Nearest Neighbor was chosen

An exact TSP solution would require evaluating an enormous number of possible routes.

For even a moderate number of destinations, the running time becomes impractical.

Nearest Neighbor provides a simple and efficient approximation that produces good routes within a very short time.

This makes it appropriate for a university routing project.

Approximation

Nearest Neighbor is not an exact algorithm.

It is a heuristic, meaning that it does not guarantee the optimal route.

Instead, it attempts to build a good solution by making the locally best decision at every step.

Consequently, the resulting route may not be the shortest possible overall route.

Complexity

Assume there are K destinations.

The algorithm computes one shortest-path search for every candidate destination during each iteration.

If SP denotes the complexity of one shortest-path computation:

$O(K^2 \times SP)$

where

- SP = O((V+E)log V) when Dijkstra is used.
- SP = O(VE) when Bellman-Ford is used.

Additional memory usage is

O(K)

for storing the route and the set of unvisited destinations.

Advantages

- Very simple to implement
- Fast for a moderate number of destinations
- Produces reasonably good routes
- Easy to combine with shortest-path algorithms

Disadvantages

- Does not guarantee the optimal route
- Early decisions may lead to a poor overall solution
- Route quality depends on the graph structure

9. Complexity Summary

Component	**Time Complexity**	**Space Complexity**
Graph construction	O(V + E)	O(V + E)
Dijkstra	O((V + E) log V)	O(V + E)
Bellman-Ford	O(VE)	O(V)
Nearest Neighbor	$O(K^2 \times SP)$	O(K)

where:

- V = number of vertices
- E = number of edges
- K = number of destinations
- SP = complexity of the shortest-path algorithm used

10. Conclusion

This project implements a routing engine capable of handling realistic road networks by considering distance, traffic conditions, weather conditions, and intersection delays when computing travel costs. The graph is represented using an adjacency list to achieve efficient memory usage and fast traversal on sparse road networks.

For shortest-path computation, the system automatically selects the most suitable algorithm. Dijkstra's algorithm is used whenever all edge weights are non-negative because it provides excellent performance. When negative edge weights are detected, the implementation switches to Bellman-Ford, ensuring correct results while also detecting negative cycles.

For routing through multiple destinations, the project applies the Nearest Neighbor heuristic. Although this approach does not always produce the optimal tour, it offers a practical approximation with significantly lower computational cost than solving the Travelling Salesman Problem exactly.